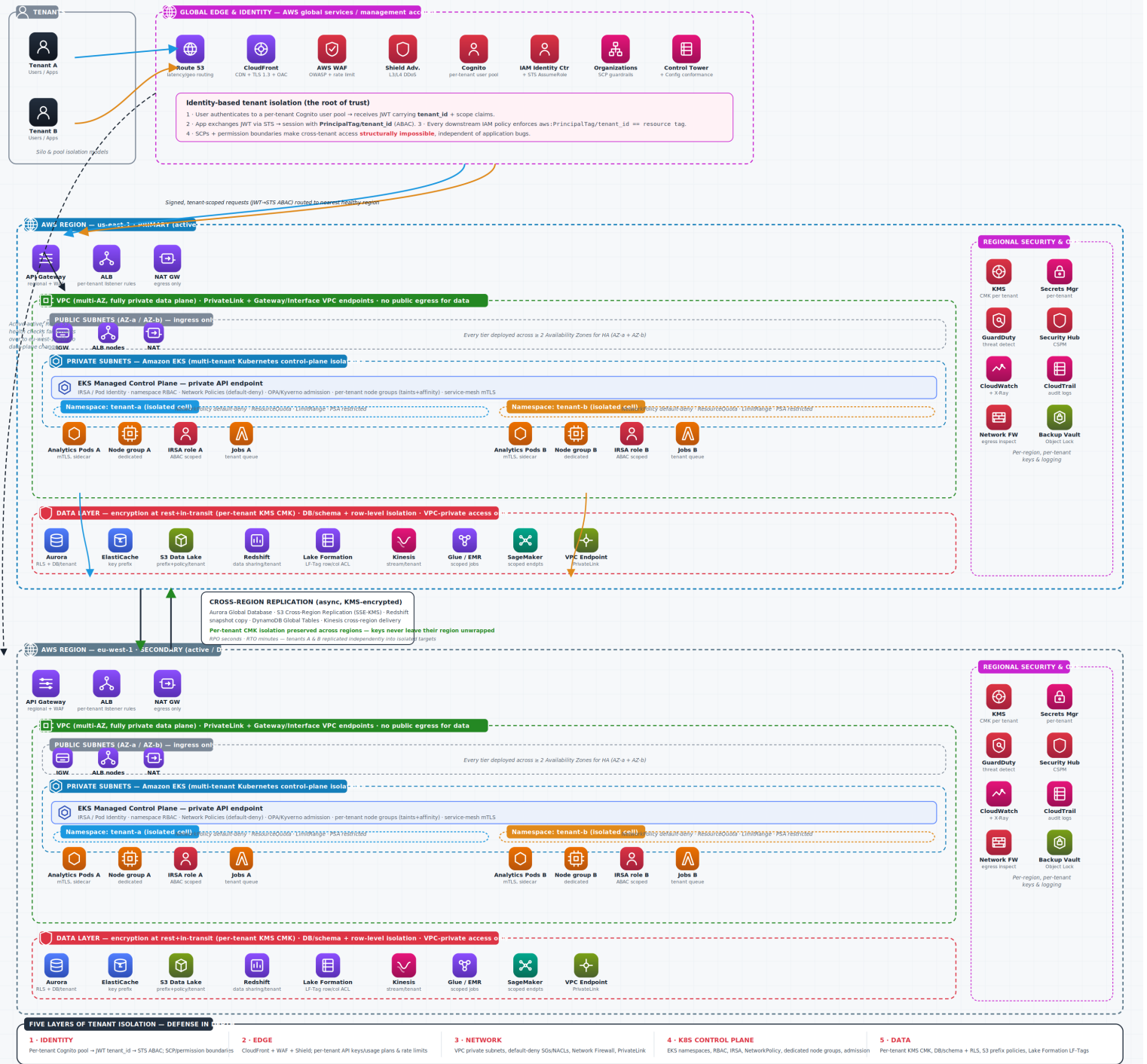


Multi-Tenant, Multi-Region SaaS Data Analytics Platform on AWS

Defense-in-depth tenant isolation across Identity · Edge · Network · Kubernetes control plane · Data — with active-active multi-region high availability and encrypted cross-region replication.



How the Architecture Works

A request-to-storage walkthrough, the isolation model at each layer, and the multi-region high-availability design.

1 The core idea: isolation is enforced, not assumed

The platform serves many tenants from shared infrastructure, but no tenant can ever reach another tenant's data — even if application code has a bug. This is achieved by making tenant identity a first-class attribute that travels with every request and is independently re-checked at five layers: **Identity, Edge, Network, the Kubernetes control plane, and Data**. Each layer can deny a cross-tenant action on its own, so a failure or misconfiguration in one layer does not collapse isolation. This is the essence of defense in depth: overlapping controls rather than a single gate.

The root of trust is identity. A tenant's `tenant_id` is established at authentication, carried in a signed token, converted into a tagged cloud session, and then enforced by attribute-based access control (ABAC) on every downstream resource. Because the enforcement lives in IAM policy and service control policies (SCPs) — not in application logic — cross-tenant access is structurally prevented at the platform level.

2 Request walkthrough: how Tenant A's data flows end to end

Follow a single analytics query from Tenant A (the blue path on the diagram). Tenant B follows an identical but separately-enforced path.

1

Authenticate (Identity). The user signs in to **Tenant A's dedicated Cognito user pool**. Cognito issues a JWT containing `tenant_id=A` plus scopes/roles. A compromised Tenant B token is meaningless here — it carries the wrong tenant claim and a different signing context.

2

Enter at the edge (Edge). The request hits **Route 53** (latency-based routing to the nearest healthy region) → **CloudFront** (TLS 1.3 termination, caching) → **AWS WAF** (OWASP rules, per-tenant rate limits) → **Shield Advanced** (L3/L4 DDoS). Per-tenant API keys and usage plans throttle noisy tenants before they reach compute.

3

Cross into the region (Network). Traffic reaches the regional **API Gateway / ALB**, which validate the JWT and apply per-tenant listener rules. Everything past this point lives in **private VPC subnets** across two Availability Zones; data services have **no public endpoints** and are reached only through VPC endpoints / PrivateLink.

4

Exchange token for a scoped session (Identity → ABAC). The application exchanges the JWT via **STS** for a temporary session whose **session tag** `aws:PrincipalTag/tenant_id = A` is set from the token claim. This tag is the linchpin: it is now attached to the caller's identity for all subsequent AWS API calls.

5

Run in an isolated namespace (Kubernetes control plane). The workload executes in the **tenant-a EKS namespace**: dedicated node group (taints/affinity), an **IRSA role** scoped to Tenant A, **default-deny NetworkPolicies**, ResourceQuotas, restricted Pod Security, and service-mesh mTLS. The pod assumes only Tenant A's IAM role — it cannot request Tenant B's role.

6

Read/write tenant data (Data). Data access is allowed only when `aws:PrincipalTag/tenant_id` matches the resource's tenant tag. Aurora enforces row-level security and per-tenant schemas/databases; S3 prefixes and bucket policies scope objects; Redshift uses per-tenant data sharing; Lake Formation applies row/column ACLs by LF-Tag. Every object is encrypted with **Tenant A's own KMS CMK**.

7

Observe & audit (cross-cutting). CloudWatch, X-Ray, CloudTrail, GuardDuty, and Security Hub record and inspect the request. Audit logs capture which tenant principal touched which resource, providing the evidence trail regulators and incident responders need.

Why this matters

Tenant B's request runs through the very same services but with `tenant_id=B`. At step 6 the IAM policy comparison (`PrincipalTag == resource tag`) fails for any Tenant A resource, so the call is denied by AWS itself. Isolation does not depend on the analytics code being bug-free.

3 The five isolation layers in detail

Each layer is independently sufficient to block a cross-tenant action. Together they form the defense-in-depth posture summarized on the diagram.

Layer	What is isolated	Primary controls	Failure contained because...
1 · Identity	Who the caller is and which tenant they represent	Per-tenant Cognito pools; JWT <code>tenant_id</code> ; STS ABAC session tags; SCPs & permission boundaries	Cross-tenant calls are denied by IAM, independent of app code
2 · Edge	Inbound traffic, abuse, and noisy-neighbor load	CloudFront + WAF + Shield; per-tenant API keys, usage plans, rate limits	A flooding/abusive tenant is throttled before reaching compute
3 · Network	Reachability between components and to data	Private subnets; default-deny SGs/NACLs; Network Firewall; PrivateLink/VPC endpoints	Data plane has no public path; lateral movement is blocked
4 · K8s control plane	Compute, runtime identity, and east-west traffic	EKS namespaces; RBAC; IRSA/Pod Identity; NetworkPolicy; dedicated node groups; OPA/Kyverno admission	A pod can assume only its own tenant role and talk only to allowed peers
5 · Data	The data itself, at rest and in queries	Per-tenant KMS CMK; Aurora RLS + DB/schema; S3 prefix policies; Lake Formation LF-Tags; Redshift data sharing	Even with access, data is encrypted per-tenant and row/cell-scoped

Silo vs. pool — the isolation spectrum

Tiers can mix models. A **pool** model shares infrastructure with logical separation (shared Aurora cluster with row-level security and ABAC) — most cost-efficient. A **silo** model dedicates resources per tenant (separate database, namespace, or even account) — strongest isolation and blast-radius control, higher cost. This design pools at the compute/edge tiers and offers silo options at the data tier for premium or regulated tenants, so the commercial tier maps directly to the isolation model.

4 Multi-region high availability

The platform runs **active-active** across two regions (us-east-1 primary, eu-west-1 secondary). Within each region every tier spans at least two Availability Zones, so an AZ failure is absorbed automatically. Across regions, **Route 53 health checks** steer tenants to the nearest healthy region; if a region degrades, traffic shifts with no data-plane change because both regions run the identical stack.

Data is kept in sync by **asynchronous, KMS-encrypted replication**: Aurora Global Database, S3 Cross-Region Replication, Redshift snapshot copy, and DynamoDB Global Tables. Crucially, **per-tenant key isolation is preserved across regions** — each region wraps data with region-local CMKs, and keys are never exported unwrapped. Typical objectives are **RPO in seconds and RTO in minutes**.

Failure scenario	Detection	Response	Tenant impact
Single AZ outage	ELB/EKS health checks	Traffic shifts to healthy AZ; pods reschedule	None (sub-minute)
Service/pod failure	Liveness/readiness probes	Kubernetes reschedules; ALB drains	None to brief retry
Full region outage	Route 53 health checks	DNS fails tenants over to secondary region	Brief; RTO minutes
Data corruption	Backups + PITR	Restore from Backup Vault (Object Lock) / PITR	Scoped to affected data

Anticipated Questions & Answers

The questions most likely to come from security, platform/SRE, and architecture review boards — with direct answers and the trade-offs behind each decision.

Security & Tenant Isolation

Q. If two tenants share an EKS cluster and an Aurora instance, what genuinely stops Tenant A from reading Tenant B's data?

Three independent mechanisms, any one of which blocks it. (1) **ABAC at the IAM layer**: the caller's STS session is tagged with its `tenant_id`, and every data resource is tagged with its owning tenant; IAM policies grant access only when the tags match, so AWS itself denies the cross-tenant call. (2) **Database-level isolation**: Aurora enforces row-level security and per-tenant schemas/databases, so even a shared connection cannot return another tenant's rows. (3) **Per-tenant KMS CMKs**: data is encrypted with the owning tenant's key, and key-use policy is also tenant-scoped, so decryption fails without the right tenant context.

The point is layering: an application bug, a misconfigured query, or a leaked connection string each only defeats one layer, not all three.

Q. How is the tenant context prevented from being spoofed or tampered with?

The `tenant_id` originates in a signed JWT from a per-tenant Cognito pool and is validated (signature, audience, expiry) at the edge and API tier. It is then mapped to an STS session tag through a trusted exchange the application cannot forge — the tag is set by the token-exchange role, not by client input. Permission boundaries and SCPs cap what any role can do, so even a compromised service role cannot escalate to set an arbitrary tenant tag or assume another tenant's role.

Q. What is the blast radius if a single pod or node is compromised?

Contained to one tenant. Pods run in per-tenant namespaces on dedicated node groups with default-deny NetworkPolicies, so a compromised pod cannot move laterally to another tenant's pods or reach data services it isn't authorized for. Its IRSA role is scoped to one tenant, so stolen credentials only unlock that tenant's resources — which are themselves encrypted with that tenant's CMK. GuardDuty (including EKS runtime monitoring) and Security Hub flag the anomaly for response.

Q. How are secrets and encryption keys managed and rotated?

Secrets live in Secrets Manager with per-tenant scoping and automatic rotation; no secrets are baked into images or environment variables — pods retrieve them at runtime via their scoped IRSA role. Encryption uses per-tenant KMS CMKs with key policies that restrict use to the owning tenant's principals; keys rotate on schedule and all key usage is logged in CloudTrail. In the secondary region, region-local CMKs are used so keys never leave their region unwrapped.

Q. How do you prove isolation to an auditor or regulator?

Through evidence, not assertions. CloudTrail records every API call with the tenant-tagged principal; Config and Security Hub continuously evaluate resources against conformance packs; Lake Formation and IAM Access Analyzer show who can access what; and Control Tower/Organizations enforce guardrails org-wide. For regulated tenants, the silo option (dedicated database or even dedicated account) provides a clean, demonstrable boundary.

Q. Why ABAC (tags) rather than a separate IAM role per tenant?

ABAC scales; per-tenant roles do not. With thousands of tenants, creating and maintaining a distinct role and policy set per tenant hits IAM limits and becomes an operational burden. ABAC uses one policy that compares the principal's tenant tag to the resource tag, so onboarding a tenant means issuing a tag, not authoring new IAM objects.

The trade-off: ABAC demands rigorous tagging discipline — every resource must be correctly tagged at creation. We enforce this with Organization tag policies, IaC guardrails, and Config rules that flag untagged resources. For the highest-assurance tenants we still fall back to silo isolation.

Scalability & Performance

Q. How does the platform scale as tenants and data volume grow?

Horizontally at every tier. EKS scales pods (HPA/KEDA) and nodes (Cluster Autoscaler/Karpenter) on demand; the data tier scales independently — Aurora adds replicas and storage auto-grows, Redshift uses elastic resize/concurrency scaling, and BigQuery-style serverless analytics (here, Athena/EMR over S3) scale per query. Because storage (S3 data lake) is decoupled from compute, you scale the two separately rather than over-provisioning whole clusters.

Q. How do you stop one tenant from starving others (the noisy-neighbor problem)?

At multiple layers. The edge applies per-tenant API usage plans and rate limits. In Kubernetes, ResourceQuotas and LimitRanges cap each namespace, and dedicated node groups keep heavy tenants off shared nodes. At the data tier, per-tenant streams/shards (Kinesis) and scoped query workloads prevent one tenant's analytics jobs from monopolizing capacity.

For tenants whose load is large or spiky enough to threaten neighbors, the answer is to move them to a **silo cell** — dedicated node group, database, or a dedicated stack — which also becomes a natural premium tier.

Q. What is the scaling unit — how do you add capacity predictably?

The **cell** (a region's full stack: VPC + EKS + data tier). When a region approaches capacity or tenant counts grow, you add cells (additional node groups, additional regions, or sharded data stores) rather than vertically scaling a single shared resource. This bounds blast radius and gives a clean unit for capacity planning and cost attribution.

Q. How does multi-region affect scalability and latency?

Active-active regions serve tenants from the nearest region (Route 53 latency routing), improving latency and spreading load. Each region scales independently, so global growth is handled by scaling regions in parallel. The cost is replication overhead and the discipline of keeping regions identical; the benefit is both lower latency and built-in capacity headroom.

Q. Does per-tenant encryption (CMK per tenant) create scaling limits?

It introduces key-management overhead and KMS request/throttling considerations at very high tenant counts and request rates. Mitigations: data-key caching to reduce KMS calls, grouping smaller tenants under shared keys where their isolation tier permits, and reserving dedicated CMKs for tenants whose compliance posture requires them. This is a deliberate cost/assurance trade-off rather than a hard ceiling.

Architectural Trade-offs

Every isolation and HA decision trades one property for another. The major ones, stated plainly:

Decision	What we gained	What it costs	When we'd choose differently
Pool by default, silo on demand	Cost efficiency at scale; simple onboarding	Logical (not physical) isolation; tagging discipline required	Regulated/large tenants → silo (dedicated DB/account)

Decision	What we gained	What it costs	When we'd choose differently
EKS over serverless-only	Portability, ecosystem, fine-grained control	Operational complexity; cluster security to manage	Small teams / simple apps → Cloud Run-style serverless
ABAC over per-tenant roles	Scales to thousands of tenants with one policy	Hard dependency on correct tagging everywhere	Few, high-assurance tenants → per-tenant roles/accounts
Active-active multi-region	Low latency + fast failover (RTO minutes)	Replication cost; must keep regions identical	Cost-sensitive / single-market → active-passive or 1 region
Per-tenant CMKs	Strong crypto isolation; clean key revocation	KMS overhead at scale; key-management ops	Many tiny tenants → shared keys where tier allows
Async cross-region replication	Performance; no write-latency penalty	Non-zero RPO (seconds of potential data loss)	Zero-RPO mandate → synchronous (latency/cost cost)

The one-paragraph summary

This is a pool-by-default, silo-on-demand SaaS platform where tenant identity is enforced — not trusted — at five independent layers, and where active-active multi-region gives low latency and minutes-level recovery. The deliberate trade-offs are operational complexity (Kubernetes, tagging discipline, replication) accepted in exchange for scalable, demonstrable isolation and resilience. The commercial tiering (standard pool vs. premium silo) maps directly onto the isolation model, so security posture and pricing reinforce each other.

Security Architecture Addendum

Principal Security Architect's review notes — design rationale, defense-in-depth controls, threat model, tenant authentication, and data residency for a global, multi-tenant data analytics product.

Framing & assumptions

This product serves tenants in multiple countries, several of whom are contractually or legally required to keep sensitive data within a specific region (GDPR, UK DPA, regional data-sovereignty regimes, sector rules such as HIPAA/PCI-DSS). The design therefore treats **data residency, tenant isolation, and auditability as first-order requirements**, not features. Throughout, controls are mapped to a STRIDE-style threat model and stated from both an attacker's and a defender's point of view. The guiding principles are **zero-trust (never trust the network), least privilege, assume-breach, and encrypt-everywhere with tenant-scoped keys**.

1 Why Route 53 over (or alongside) Global Accelerator

Short answer: they solve different problems, and for an HTTP(S) analytics product the primary entry should be **CloudFront + Route 53**, with **Global Accelerator (GA)** reserved for the non-HTTP or ultra-low-latency paths. They are complementary, not mutually exclusive.

Dimension	Route 53 (+ CloudFront)	Global Accelerator	Decision for this product
Layer / protocol	DNS (L7 routing) + CDN for HTTP(S)	L3/L4 anycast TCP/UDP	Tenant traffic is HTTPS APIs/dashboards → CloudFront+R53 fits
Edge security	WAF + Shield at CloudFront edge; TLS termination, OAC	Shield only; no native WAF/L7 inspection	We need L7 WAF on tenant APIs → CloudFront
Caching	Yes — offloads origin, lowers egress	No caching	Dashboards/static assets benefit from CDN caching
Failover model	DNS health-check failover (TTL-bound)	Instant anycast failover; static anycast IPs	GA wins on sub-minute failover & fixed IPs
Client IP allow-listing	Edge IPs change	2 static anycast IPs (allow-list friendly)	Use GA when enterprise tenants must allow-list our IPs
Data residency control	Geo/latency & geolocation routing per tenant	Routes to nearest healthy endpoint	R53 geolocation lets us pin EU tenants to EU region

Where Global Accelerator does earn its place:

- **Enterprise tenants that must IP-allow-list us** — GA's two static anycast IPs are stable allow-list targets, unlike rotating CDN edge ranges.
- **Non-HTTP ingestion** — if tenants stream telemetry over raw TCP/UDP or gRPC at scale, GA fronts the NLB far better than a CDN can.
- **Faster regional failover** — GA shifts traffic at the network layer in seconds, avoiding DNS TTL/caching lag during a region evacuation.

My recommendation

Default ingress = **Route 53 (geolocation + latency) → CloudFront (WAF, Shield Advanced, TLS 1.3, OAC) → regional ALB**. Add **Global Accelerator in front of an NLB** as a second, opt-in entry for (a) enterprise tenants needing static-IP allow-listing and (b) high-throughput non-HTTP ingestion. Critically, Route 53 **geolocation routing is also a data-residency control**: EU-resident tenants are resolved only to EU endpoints, so traffic never transits a non-permitted region.

2 Additional controls at the compute and network layers

Beyond what the diagram shows, here is the fuller control set I would mandate. The principle is layered enforcement: SCP → VPC → subnet (NACL) → instance/ENI (SG) → host → pod → runtime.

Network-layer controls

Control	What it does	Attacker behavior it stops
NACLs (subnet, stateless)	Coarse default-deny at subnet edges; explicit deny for known-bad CIDRs & egress ports	Blunt-force lateral movement / scanning across subnets even if an SG is misconfigured
Security Groups (stateful, per-ENI)	Least-privilege, reference-by-SG (not CIDR); no 0.0.0.0/0 inbound	Direct reach to data ports (5432/6379/etc.) from anything but the app tier
AWS Network Firewall	Stateful L3-L7 egress inspection, domain allow-lists, IDS/IPS (Suricata rules)	C2 callbacks & data exfiltration to unapproved domains
VPC endpoints / PrivateLink	Private reach to S3/KMS/etc.; no internet path for data	Exfil via public service endpoints; SSRF pivoting to AWS APIs
VPC Service Controls pattern (SCP + endpoint policy)	Bind data access to our VPCs/accounts only	Use of stolen creds from outside our network perimeter
Egress via Cloud NAT + deny-by-default	All outbound brokered & logged; default route is deny	Silent outbound exfiltration from a compromised workload
DNS Firewall (Route 53 Resolver)	Block resolution of malicious / DGA / data-tunneling domains	DNS-based C2 and DNS-tunneling exfiltration
No public IPs on data/compute	Workloads on private subnets; admin via SSM Session Manager + IAP	Internet-facing brute force; exposed management ports

Compute / Kubernetes-layer controls

Control	What it does	Attacker behavior it stops
No SSH; SSM Session Manager only	Audited, IAM-gated, keyless shell access	Stolen SSH keys; unlogged interactive access
Immutable, hardened AMIs/images (CIS)	Golden images, read-only root FS, signed	Persistence via host tampering; drift
EKS: PSA 'restricted', no privileged pods	Blocks host mounts, hostNetwork, privilege escalation	Container breakout to the node

Control	What it does	Attacker behavior it stops
Admission control (Kyverno/OPA) + Binary Authorization	Only signed, scanned images from our registry run	Supply-chain / poisoned-image deployment
IRSA / Pod Identity (no node-wide creds)	Each pod gets only its tenant-scoped role	Credential theft granting node- or cross-tenant access
Falco / GuardDuty EKS Runtime Monitoring	Detects shell-in-container, crypto-mining, syscalls	Live intrusion & post-exploitation activity
seccomp / AppArmor profiles	Restrict syscalls available to containers	Kernel-exploit breakout attempts
Dedicated node groups per tenant tier	Hard compute boundary for silo/premium tenants	Noisy-neighbor + cross-tenant side-channel risk
IMDSv2 enforced, hop-limit 1	Blocks SSRF-to-credential theft from metadata	SSRF stealing the instance role (classic cloud breach)
Ephemeral nodes + frequent rotation	Cattle-not-pets; short-lived hosts	Long-term attacker persistence

3 Logging, detection, and automated remediation of anomalies

The model is a closed loop: **collect everything centrally and immutably** → **detect with managed + custom analytics** → **triage and correlate** → **respond automatically where safe, with human-in-the-loop for destructive actions**. Logs are the source of truth and are protected from the tenants and from operators alike.

Collect (telemetry, centralized & tamper-evident)

- **CloudTrail (all regions, org-level, incl. data events)** for S3/Lambda → a dedicated, locked *log-archive* account; **S3 Object Lock (WORM)** + MFA-delete so logs can't be altered, even by an admin.
- **VPC Flow Logs, Network Firewall logs, DNS Resolver query logs, ALB/CloudFront/WAF logs** for full network visibility.
- **EKS control-plane audit logs, application logs, and Kubernetes audit** shipped to the same archive; **every log line carries the tenant_id** for per-tenant forensics.
- **Config** records every resource configuration change (the 'what changed and when' timeline).

Detect (managed services + custom analytics)

Signal source	Detects	Example anomaly
GuardDuty (+ EKS/RDS/S3 protection, Malware Protection)	Account/network/runtime threats via ML + threat intel	Credential exfil, crypto-mining, anomalous API geography
Security Hub (+ conformance packs)	Posture drift & aggregated findings vs. CIS/PCI	A bucket made public; an SG opened to 0.0.0.0/0
Detective	Graph-based root-cause & blast-radius	'Which identity, from where, touched what'
IAM Access Analyzer	External / cross-account / cross-tenant access paths	A resource policy that exposes a tenant's data

Signal source	Detects	Example anomaly
Macie	Sensitive-data discovery in S3 (PII/PHI)	Unencrypted PII landing in the wrong prefix/region
Custom CloudWatch / OpenSearch detections	Tenant-aware behavioral rules	Tenant A principal querying Tenant B's tagged data

Respond & remediate (graduated, mostly automated)

Findings flow into **EventBridge**, which triggers **auto-remediation via Lambda / SSM Automation / Config remediation**. Actions are graduated by severity and reversibility:

Anomaly	Automated response (now)	Human-in-the-loop (if destructive)
Public S3 bucket detected	Config rule auto-reverts to private; block-public-access re-applied	Notify owner; ticket for root-cause
Leaked/abused IAM credential	Revoke sessions, attach deny-all, rotate key (SSM)	SecOps confirms before disabling the principal
Compromised pod/node (runtime alert)	Cordon+isolate node via NetworkPolicy; snapshot for forensics	Approve termination & redeploy from golden image
Cross-tenant access attempt	Block principal, page on-call, freeze the session	Incident bridge; tenant notification per contract
DDoS / volumetric	Shield Advanced + WAF rate rules auto-engage	SRT engagement for large events
Anomalous data egress	Network Firewall blocks destination; quarantine workload	Forensic review before restoring egress

Operating model

All findings are normalized into **Security Hub** and a SIEM (OpenSearch / Splunk / Google SecOps) for correlation, with **defined SLAs by severity** (e.g. critical: auto-contain in seconds, human ack < 15 min). We run **game-days and purple-team exercises** quarterly to validate that detections fire and runbooks work. Mean-time-to-detect and mean-time-to-contain are tracked as board-level metrics.

4 How tenants authenticate and gain access — securely

This is the section for leadership: how a tenant's **users** and **applications/services** prove who they are and get scoped, time-bound access — with zero standing credentials wherever possible.

A. Human users (tenant employees using our dashboards)

- Each tenant has a **dedicated identity boundary** (Cognito user pool, or federation to *their* IdP via SAML/OIDC — Okta, Entra ID, Google). Enterprises keep using their own SSO; we never store their passwords.
- **MFA is mandatory** (phishing-resistant WebAuthn/passkeys preferred); conditional access on device posture, geography, and risk score.
- On login the user receives a **short-lived OIDC token carrying tenant_id, roles, and scopes**. Tokens are minutes-lived; refresh is bounded and revocable.
- The token is exchanged via **STS** for a session whose **PrincipalTag/tenant_id drives ABAC** on every downstream call — no long-lived AWS keys ever reach a user.

B. Tenant applications / machine-to-machine

- App clients use **OAuth2 client-credentials** with rotating secrets in Secrets Manager, or — preferred — **mTLS / signed requests** and **OIDC workload federation** so no static secret exists.
- For partner/SaaS integrations, **IAM Roles Anywhere** or **workload identity federation** issue short-lived credentials against an X.509 / OIDC trust — again, zero standing keys.
- Every API call is authorized at the gateway (validate signature, audience, expiry, tenant claim) and re-authorized at the resource by ABAC.

The one-sentence version for executives

Every tenant — user or app — authenticates against an isolated, MFA-protected identity boundary, receives a **short-lived, tenant-scoped token**, and that token is converted into **temporary, attribute-based AWS access** that AWS itself confines to that tenant's resources; **there are no shared or long-lived credentials**, and access is fully audited per tenant.

5 Data residency & sovereignty (global tenants)

Because tenants in different countries must keep sensitive data in-region, residency is enforced technically, not by policy alone:

- **Region pinning per tenant** — a tenant's home region is metadata; Route 53 geolocation resolves them only to that region, and their data stores live only there.
- **SCP / Organization region-deny guardrails** — services cannot create data resources outside permitted regions, even by mistake.
- **Replication scoped to compliant regions** — e.g. an EU tenant replicates EU→EU only; cross-bloc replication is blocked by policy and key locality.
- **Region-local KMS CMKs** — keys never leave their region unwrapped, so even backups/replicas remain sovereign; optionally External Key Store (XKS) for hold-your-own-key.
- **Data classification + Macie** — detect sensitive data that lands in the wrong region/prefix and auto-quarantine.

- **Assured/sovereign options** for the strictest tenants — dedicated accounts, personnel-access controls, and evidence for auditors.

6 Threat model — attacker vs. defender, stage by stage

Thinking as the attacker first, then the defender. Mapped loosely to STRIDE and the kill-chain; each row is a concrete way in and the control(s) that close it. These are the concerns I want the review board to interrogate.

Edge & identity

Attacker's move (threat)	Defender's control(s)
Credential stuffing / phishing tenant users (Spoofing)	Mandatory phishing-resistant MFA (passkeys); Cognito advanced security; WAF bot control; impossible-travel detection
Stolen/replayed JWT or token forgery (Spoofing/Tampering)	Short token TTLs; signature+audience+expiry validation; key rotation; token-binding; revocation on risk signal
DDoS to deny service (DoS)	Shield Advanced; CloudFront/WAF rate limiting; autoscaling; GA anycast absorption
Enumerating tenants / IDOR across tenant_id (Info disclosure)	Server-side ABAC on every object; opaque IDs; deny-by-default; access-pattern anomaly detection

Network & compute

Attacker's move (threat)	Defender's control(s)
SSRF to steal instance metadata creds (Elevation)	IMDSv2 + hop-limit 1; egress allow-lists; no instance-profile over-grants
Lateral movement after a foothold (Elevation)	Default-deny NACLs/SGs/NetworkPolicy; micro-segmentation; PrivateLink; no flat network
Container breakout to node (Elevation)	PSA restricted; seccomp/AppArmor; no privileged pods; runtime monitoring (Falco/GuardDuty)
Data exfiltration to attacker infra (Info disclosure)	Network Firewall + DNS Firewall egress control; VPC endpoints; flow-log anomaly detection

Data & supply chain

Attacker's move (threat)	Defender's control(s)
Read another tenant's data (Info disclosure)	ABAC tag-match deny; Aurora RLS; per-tenant CMK; Lake Formation row/col ACLs
Poisoned dependency / malicious image (Tampering)	Signed images; Binary Authorization; SCA/Inspector scanning; SLSA provenance
Malicious/negligent insider or operator (Elevation/Repudiation)	Break-glass with approval; SSM-only access fully logged; WORM logs admins can't alter; separation of duties
Ransomware on data/backups (Tampering/DoS)	Immutable backups (Object Lock) in isolated account; versioning; tested restores; least-privilege delete

Residual risks I'm explicitly tracking

(1) **Tagging discipline** is load-bearing for ABAC — mitigated by tag-policy enforcement, IaC guardrails, and Config rules, with silo fallback for the highest-assurance tenants. (2) **Identity-provider compromise on the tenant side** — mitigated by our own conditional access and anomaly detection, but partially outside our control. (3) **Novel zero-days in shared services** — mitigated by assume-breach segmentation, fast patching, and blast-radius containment. These are reviewed each quarter and reported up.

7 Summary for the review board

This design assumes breach and makes a single compromise **survivable and contained**: identity is enforced cryptographically and re-checked at five layers; the network is zero-trust with default-deny and inspected egress; compute is hardened, keyless, and runtime-monitored; data is encrypted with per-tenant, region-local keys and never leaves its sovereign region; and everything is logged immutably with automated, graduated response. The deliberate cost is operational rigor — tagging discipline, key management, and replication overhead — which we accept in exchange for isolation we can **prove** to tenants and regulators. The two highest-leverage investments I'm asking for are **phishing-resistant MFA everywhere** and a **mature detect-and-auto-remediate pipeline**, because they neutralize the two most common real-world breach paths: stolen credentials and slow response.